Cocktail-Chatbot – Dokumentation

Stand: 12. Oktober 2026 · Bausteine: Rasa, Rasa-Action-Server (Python), Node-RED, Excel-Datei

1. Worum geht es?

Der Bot schlägt Cocktails vor und verrät Rezepte. Man kann ihm zwei Dinge sagen:

1. **Welche Zutaten man zu Hause hat** („Ich habe Rum und Limette“) → er nennt passende Cocktails.
2. **Welchen Cocktail man nehmen möchte** („Mojito“) → er nennt die Zutaten.

Entstanden ist er als eigenes Projekt zum Webscraping-/Wetter-Chatbot-Kurs. Gechattet wird im Terminal (`rasa shell`), es gibt keine Weboberfläche.

2. Architektur im Überblick

Drei Programme sprechen miteinander, die Daten liegen in einer Excel-Datei:

```
Nutzer  (Terminal in VS Code: rasa shell)
   |  Text
   ▼
Rasa  (Port 5005)
   NLU: erkennt Intent + Entities (z. B. Zutaten)
   Dialog: Story bestimmt die nächste Action
   |  HTTP, Port 5055
   ▼
Rasa-Action-Server  (rasa run actions, actions.py)
   |  HTTP POST, Port 1880
   ▼
Node-RED  (Endpunkte /cocktail_lookup und /zutaten_lookup)
   hält eine Zutaten-Map im Speicher (global context)
   |  einmalig eingelesen aus
   ▼
Cocktail_Test.xlsx
```

Baustein	Port	Aufgabe
Rasa (`rasa shell`)	5005	Sprache verstehen, Dialog steuern
Action-Server (`rasa run actions`)	5055	Python-Code, der auf Rasas Anweisung läuft
Node-RED	1880	Cocktail-Daten bereitstellen

Merksatz: Rasa versteht, der Action-Server fragt, Node-RED weiß.

3. Starten

Jeweils eigenes Terminal in VS Code, in jedem zuerst die Python-Umgebung aktivieren (z. B. `conda activate <envname>`) und in den Rasa-Projektordner wechseln.

1. **Node-RED:** `node-red` (Editor im Browser: http://localhost:1880)
2. **Action-Server:** `rasa run actions`
3. **Chat:** `rasa shell` → beim Prompt `Your input ->` schreiben

Die Reihenfolge ist wichtig: Der Action-Server ruft Node-RED auf, Node-RED muss also schon laufen.

Wann muss neu trainiert werden?

- Änderungen an `nlu.yml` , `domain.yml` , `stories.yml` oder `config.yml` → `rasa train` , danach `rasa shell` neu starten.
- Änderungen an `actions.py` → nur den Action-Server neu starten (kein Training nötig).
- Änderungen an Node-RED → im Editor auf „Deploy“ klicken.

4. Die Bausteine im Detail

4.1 Datenquelle: Cocktail_Test.xlsx

Ein Tabellenblatt („Tabelle1“) mit rund 120 Cocktails. Spalten:

- `Cocktail` – Name
- `Spirituose` , `Alkoholische Aromazutat` , `Verlängerer` , `Filler` , `Frucht` , `sonstige Zutaten` , `Eis` – Zutaten nach Rolle sortiert
- `Kategorie` – z. B. Sours, Longdrink, Aperitif, Mule

Mehrere Zutaten in einer Zelle stehen durch Komma oder Semikolon getrennt, oft mit Mengenangabe in Klammern, z. B. `Bourbon (4cl)` oder `Scotch (1cl); Bourbon (2cl)` .

4.2 Node-RED (`flows.json`)

Der Flow (Tab „Flow 4“) hat drei Teile. Die Exportdatei `flows(1).json` ist vollständig.

Teil 1 – Daten laden (automatisch beim Start)

Der Inject-Node „Excel laden“ löst 0,1 Sekunden nach dem Start von Node-RED einmal aus. Der Node „xlsx to json“ (Zusatzpaket `node-red-contrib-xlsx-to-json`, muss installiert sein) liest `Cocktail_Test.xlsx`, Blatt „Tabelle1“. Der Dateipfad ist fest eingetragen: `C:\Users\fande\Documents\Freizeit\Cocktail_Test.xlsx`. Die Zeilen gehen weiter an **„function 1“**, die Folgendes macht:

1. Nimmt die Excel-Zeilen als JSON-Array (`msg.payload`).
2. Sammelt pro Cocktail alle Werte aus den Spalten Spirituose, Alkoholische Aromazutat, Verlängerer, Filler, Frucht, sonstige Zutaten, Eis und Kategorie (getrennt bei Komma oder Semikolon, kleingeschrieben).
3. Baut daraus eine **Zutaten-Map**: Schlüssel = einzelne Zutat (oder Kategorie), Wert = Liste der Cocktails, die sie enthalten (jeweils mit Name und kompletter Zutatenliste).
4. Speichert die Map im **global context** unter `zutatMap`.

Teil 2 – Zutaten → Cocktails (`POST /cocktail_lookup`)

Ablauf: `http in` → `function 3` → `http response` (die Antwort geht zurück an Rasa).

- Nimmt `payload.zutaten` (Liste) und macht alles klein.
- Sucht pro Zutat alle Map-Schlüssel, die den Suchbegriff **enthalten** (Teiltext-Suche): „rum“ trifft also auch „weißer rum (4cl)“.
- Bildet die **Schnittmenge**: Es kommen nur Cocktails zurück, die *alle* genannten Zutaten enthalten (UND).
- Antwort: `{"cocktails": [{"name": …, "zutaten": […]}]}`. Ohne Treffer ist die Liste leer. Ist die Map noch nicht geladen, kommt zusätzlich der Text „Die Zutatendaten sind noch nicht geladen.“

Teil 3 – Cocktail → Zutaten (`POST /zutaten_lookup`)

Ablauf: `http in` → `function 2` → `http response`.

- Nimmt `payload.cocktail` und vergleicht ihn **exakt** (Groß-/Kleinschreibung egal) mit den Cocktailnamen.
- Antwort: `{"name": …, "zutaten": […]}`, sonst nur `{"text": …}` ohne `zutaten`.

Testen ohne Rasa: Zwei Inject-Nodes schicken `{"zutaten": ["Longdrink"]}` an function 3 bzw. `{"cocktail": "Moscow Mule"}` an function 2. Das Ergebnis erscheint in der Debug-Seitenleiste. Die deaktivierten Debug-Nodes mit `payload.tracker…` stammen vermutlich aus dem früheren Versuch, Rasa direkt mit Node-RED zu verbinden.

4.3 Rasa-Projekt

Datei	Inhalt
`nlu.yml`	Trainingsbeispiele für zwei Intents, Entities sind in der Form `[Rum](zutaten)` markiert
`domain.yml`	Intents, Entities, Slots, Actions und die Antwort `utter_default`
`stories.yml`	Der Dialogablauf (eine Story, siehe unten)
`config.yml`	Sprache `de`; Pipeline und Policies sind `null`, Rasa nimmt also die Standardkonfiguration (im Kommentar der Datei aufgelistet)
`endpoints.yml`	Adresse des Action-Servers: `http://localhost:5055/webhook` (die auskommentierte Zeile darüber zeigt einen früheren Versuch direkt auf Node-RED)
`test_stories.yml`	Die unveränderte Rasa-Vorlage (greet, mood …), passt nicht zu diesem Bot

Intents

- `suche_cocktail` – der Nutzer nennt Zutaten. Entity: `zutaten`. Über 100 Beispielsätze.
- `cocktail_aussuchen` – der Nutzer wählt einen Cocktail. Entity: `cocktail`. Etwa 60 Beispielsätze.

Slots

- `zutaten` (Liste) – wird aus der Entity `zutaten` gefüllt und von der ersten Action gelesen.
- `cocktail_name` (Text) – wird aus der Entity `cocktail` gefüllt, aktuell aber von keiner Action benutzt.

Story (der gesamte Dialogablauf):

```
intent: suche_cocktail      → action_suche_cocktail
intent: cocktail_aussuchen  → action_suche_rezept
```

4.4 Action-Server (`actions.py`)

`action_suche_cocktail`

- Liest den Slot `zutaten`, macht alles klein (`.lower()`), damit es zur Schreibweise in der Zutaten-Map passt.
- Ohne Zutaten antwortet der Bot: „Bitte nenne mir mindestens eine Zutat."
- Sendet `POST http://localhost:1880/cocktail_lookup` mit `{"zutaten": [...]}` (Timeout 5 Sekunden).
- Erwartet als Antwort `{"cocktails": [{"name": ...}, ...]}` und formuliert daraus einen Satz („A, B und C. Welchen möchtest du?").
- Ohne Treffer: „Leider habe ich keinen Cocktail mit diesen Zutaten gefunden."

`action_suche_rezept`

- Nimmt die **komplette letzte Nutzernachricht** (`tracker.latest_message['text']`), kleingeschrieben – nicht den Slot `cocktail_name`.
- Sendet `POST http://localhost:1880/zutaten_lookup` mit `{"cocktail": "<Nachricht>"}`.
- Erwartet `{"zutaten": [...], "name": ...}` und antwortet: „Für einen <Name> brauchst du: …"
- Ohne Treffer: „Ich habe leider kein Rezept für … gefunden."

5. Beispielablauf

1. Nutzer: *„Ich habe Rum und Limette"*
2. Rasa erkennt Intent `suche_cocktail`, Entities `zutaten` = Rum, Limette → Slot `zutaten` ist gesetzt.
3. Die Story ruft `action_suche_cocktail` auf (Rasa → Action-Server, Port 5055).
4. Die Action schickt `{"zutaten": ["rum", "limette"]}` an Node-RED.
5. Node-RED schlägt in der Zutaten-Map nach und antwortet mit einer Cocktailliste.
6. Bot: *„Mit diesen Zutaten kannst du folgende Cocktails machen: … Welchen möchtest du?"*
7. Nutzer: *„Mojito"* → Intent `cocktail_aussuchen` → `action_suche_rezept`.
8. Die Action schickt den Text an `/zutaten_lookup`, Node-RED liefert die Zutatenliste.
9. Bot: *„Für einen Mojito brauchst du: …"*

6. Typische Fehler

Meldung / Symptom	Ursache	Lösung
`error while attempting to bind on address ('0.0.0.0', 5005)`	Port 5005 ist belegt: `rasa shell` und `rasa run --enable-api` laufen nicht gleichzeitig	Nur eines von beiden starten, oder `rasa shell --port 5006`. Alten Prozess finden: `netstat -ano | findstr :5005`, dann `taskkill /PID <PID> /F`
Verbindungsfehler im Action-Server-Terminal (Connection refused, Port 1880)	Node-RED läuft nicht	Node-RED zuerst starten
Bot findet nie etwas („keinen Cocktail gefunden“)	Zutaten-Map ist leer, weil die Excel-Datei nicht gelesen werden konnte (der Pfad im Node „xlsx to json“ ist fest eingetragen und gilt nur auf dem ursprünglichen Rechner), oder die Schreibweise passt nicht zur Map	Pfad im Node prüfen, den Inject „Excel laden“ von Hand auslösen und im Debug-Fenster prüfen, was in `zutatMap` steht
`rasa: command not found`	Falsche oder keine Python-Umgebung aktiv	Umgebung aktivieren
„No model found“	Noch nicht trainiert	`rasa train`

7. Bekannte Schwachstellen und Ideen

Beim Durchgehen der Dateien sind mir diese Punkte aufgefallen. Sie sind der Startpunkt für die Weiterentwicklung.

Daten und Suche

- **Kategorien stehen in der Zutatenliste.** `function 1` liest auch die Spalte `Kategorie`. Dadurch funktioniert die Suche nach „Longdrink“ oder „Sours“, aber die Rezeptantwort nennt die Kategorie dann als Zutat. Sauberer: Kategorien in einem eigenen Feld speichern.

- **Die Teiltext-Suche trifft zu viel.** `function 3` sucht mit `includes`: „gin“ findet auch „ginger ale/beer“, also werden bei Gin auch Mules vorgeschlagen. Besser: ganze Wörter vergleichen.
- **Mengenangaben stecken in den Schlüsseln** (z. B. `weißer rum (4cl)`). Für die Rezeptanzeige ist das gewollt, für die Suche stört es. Idealerweise zwei Felder: Suchname und Anzeigetext.
- In der Excel heißt die Spaltenüberschrift `Eis ` (mit Leerzeichen am Ende), im Function-Node wird `Eis` abgefragt; die Spalte wird deshalb vermutlich nicht gelesen.
- Datenpflege: Tippfehler (z. B. „Scoth“, „Organenlikör“, „Angustura“), unterschiedliche Schreibweisen und einzelne Zeilen ohne Zutaten (z. B. B52, Singapur Sling).

Rasa

- **Wichtigster Punkt: `/zutaten_lookup` vergleicht exakt.** `action_suche_rezept` schickt die ganze Nutzernachricht, Node-RED vergleicht sie mit den Cocktailnamen. „Mojito“ funktioniert, „Ich möchte den Mojito“ nicht. Besser: den Slot `cocktail_name` übergeben.
- In `nlu.yml` stehen bei den Kategorie-Beispielen Annotationen mit Leerzeichen, z. B. `[Aperitif] (zutaten)`. So wird die Entity vermutlich nicht erkannt; richtig ist `[Aperitif](zutaten)`. Die Markierung als `zutaten` passt inhaltlich, weil Kategorien in der Zutaten-Map stehen.
- Es gibt nur eine Story, keine Begrüßung, Verabschiedung oder Hilfe und keine hochgeladene `rules.yml`. Der Bot kennt nur `utter_default`.
- `test_stories.yml` ist die Standardvorlage und passt nicht zu Intents und Actions dieses Bots. `rasa test` ist deshalb noch nicht sinnvoll nutzbar.
- `config.yml` ist leer (`null`), die Standardpipeline wird verwendet. Für die Weiterentwicklung (z. B. Synonyme, Lookup-Tabellen für Zutaten) lohnt sich eine explizit hinterlegte Pipeline.

Action-Server

- Kein Fehlerhandling: Läuft Node-RED nicht, bricht die Action mit einer Python-Fehlermeldung ab (`try/except` um `requests.post` mit freundlicher Antwort wäre einfach).

8. Dateiübersicht

Datei	Ort / Zweck
`actions.py`	Rasa-Projekt, Ordner `actions/` – Python-Aktionen

Datei	Ort / Zweck
`domain.yml`, `config.yml`, `endpoints.yml`	Rasa-Projekt, Hauptordner
`nlu.yml`, `stories.yml`	Rasa-Projekt, Ordner `data/`
`test_stories.yml`	Rasa-Projekt, Ordner `tests/`
`flows.json`	Export des Node-RED-Flows
`Cocktail_Test.xlsx`	Cocktail-Daten